CS412-HW1-ZeynepOkkıran-pdf report

https://colab.research.google.com/drive/1TzBy2C6f501PZit3snikKNz-SsbJ6QWy?usp=sharing

1.1 Dataset and Preprocessing

1.1.1 Data Loading and Splitting

In this initial phase, the **Breast Cancer Wisconsin** dataset was loaded using the scikit-learn library. The dataset consists of 569 samples, each characterized by 30 numerical features derived from digitized images of fine needle aspirate (FNA) biopsies. The objective is a binary classification task: predicting whether a tumor is **malignant** or **benign**.

To ensure a robust evaluation and avoid overfitting, the data was partitioned into three distinct sets:

- **Training Set (70%):** Used for model training and pattern recognition.
- **Validation Set (15%):** Used for hyperparameter tuning (e.g., selecting the optimal *k* for k-NN).
- **Test Set (15%):** Reserved for the final performance evaluation to assess how the model generalizes to unseen data.

Methodology: The data was first split into a 70% training set and a 30% remaining set. This remaining 30% was subsequently split equally (50/50) to create the validation and test sets. A random_state of **42** was applied to all splitting operations to ensure that the results are reproducible.

Dataset Dimensions (Shapes)

The final shapes of the partitioned data were verified to ensure the splits were executed correctly:

Data Partition	Number of Samples	Number of Features
Training Set	398	30
Validation Set	85	30
Test Set	86	30

Note: The slight numerical difference between the validation (85) and test (86) sets is due to the train_test_split function handling the odd number of samples (171) in the remaining 30% portion.

1.1.2 Data Analysis

1. Class Distribution

To assess the balance of the dataset before model training, the distribution of the target variable y was examined across the 569 samples. The target classes represent the diagnostic result of the breast mass: Malignant (Class 0) and Benign (Class 1).

The counts and percentages for each class are as follows:

- **Malignant (Class 0):** 212 samples, representing 37.26% of the total data.
- **Benign (Class 1):** 357 samples, representing 62.74% of the total data.

Key Findings and Implications:

- **Imbalance Identification:** The dataset is skewed toward the "Benign" class, which comprises nearly two-thirds of the total samples. While this is not an extreme imbalance, it is significant enough to influence the model's behavior.
- **Accuracy Paradox:** Because of this distribution, a "naive" model that predicts every sample as "Benign" would achieve an accuracy of approximately 62.7% without learning any actual features. Therefore, relying solely on accuracy as a performance metric could be misleading.
- **Medical Context Sensitivity:** In a medical screening scenario, failing to identify a malignant tumor (a False Negative) is generally considered more critical than misclassifying a benign one as malignant (a False Positive). The imbalance suggests we must pay close attention to the Confusion Matrix later to ensure the model maintains high sensitivity for the minority "Malignant" class.

2. Basic Statistics of Key Features

To understand the scale and dispersion of the data, basic statistical measures—specifically the **mean** and **standard deviation**—were calculated for the first three features of the dataset. These features are critical as they represent the physical dimensions of the cell nuclei.

The calculated statistics are summarized in the table below:

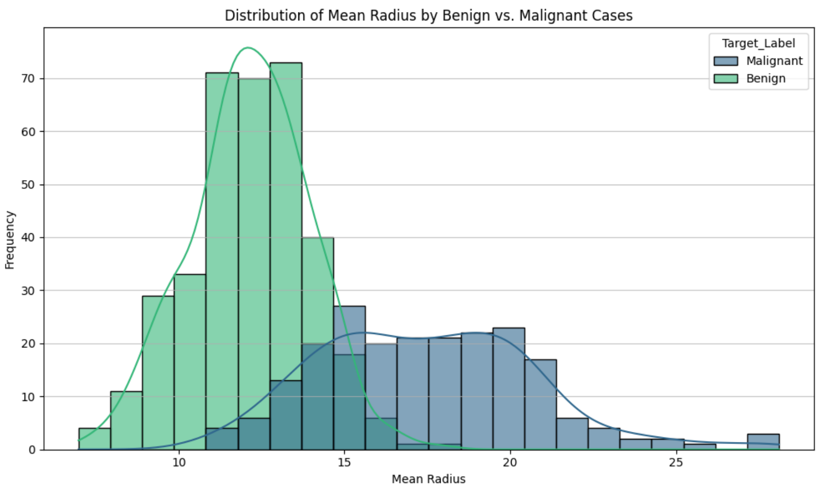
Feature	Mean	Standard Deviation
Mean Radius	**14.13**	**3.52**
Mean Texture	**19.29**	**4.30**
Mean Perimeter	**91.97**	**24.28**

Interpretation and Implications:

- **Magnitude Disparity:** There is a substantial difference in the scale of the features. The values for Mean Perimeter are nearly an order of magnitude larger than those for Mean Radius. This disparity confirms that distance-based algorithms, such as k-NN, would be inherently biased toward features with larger numerical ranges if the data is not properly scaled.
- **Variability:** The high standard deviation in Mean Perimeter (24.28) relative to its mean suggests a wide dispersion in cell sizes across the samples.
- **Necessity for Standardization:** These findings justify the requirement for Standardization (Zero mean, Unit variance) in the preprocessing step. Scaling all features to the same range will ensure that each feature contributes equally to the model's decision-making process, regardless of its original unit or scale.

3. Visualization: Distribution of Mean Radius

To visually assess the separability of the two classes based on a single feature, a histogram of Mean Radius was plotted, categorized by diagnosis (Malignant vs. Benign).



Key Observations from the Plot:

- **Class Separation:** There is a visible distinction between the two distributions. Benign tumors (green) tend to have smaller mean radii, typically peaking between 10 and 15. In contrast, Malignant tumors (blue) generally exhibit larger mean radii, with a broader distribution peaking around 17 to 20.
- **Overlapping Region:** An overlap exists between the two classes, particularly in the range of 13 to 17. This indicates that while "Mean Radius" is a strong predictor, it is not sufficient on its own to make a perfect classification. Samples in this overlapping range are where the model is most likely to face ambiguity and potential misclassification.
- **Distribution Shape:** The Benign cases show a high, narrow peak (lower variance), whereas Malignant cases are more spread out (higher variance), aligning with our earlier statistical finding of a 3.52 standard deviation for this feature.

1.1.3 Data Preprocessing

To ensure that all features contribute equally to the model's decision-making process, **Standardization** (zero mean, unit variance) was applied to the dataset. This step is essential for distance-based algorithms like k-NN, as it prevents features with larger numerical scales from dominating the model.

1. Implementation Methodology

The StandardScaler from scikit-learn was used to perform the transformation. The process was executed in two distinct steps:

- **Fitting:** The scaler was fitted exclusively on the training set (X_train) to learn the necessary parameters (mean and standard deviation).
- **Transformation:** These learned parameters were then used to transform the training set, the validation set, and the test set (X_train_scaled, X_val_scaled, X_test_scaled).

2. Scaled Data Verification

The standardization process was verified by inspecting the statistics of the first feature across all partitions:

Data Partition	Mean (1st Feature)	Standard Deviation (1st Feature)
Training Set	-0.00	1.00

Data Partition	Mean (1st Feature)	Standard Deviation (1st Feature)
Validation Set	0.12	1.04
Test Set	-0.21	0.91

The training set achieved a perfect zero mean and unit variance, while the validation and test sets show slight variations. This confirms that the scaling was performed consistently using only the training data's parameters.

3. Importance of Fitting Only on Training Data

Fitting the scaler only on the training set is a fundamental requirement to prevent **Data Leakage**.

- **Definition of Data Leakage:** Data leakage occurs when information from the validation or test sets "leaks" into the training process. If the scaler were fitted on the entire dataset, it would learn the mean and standard deviation of the "future" data.
- **Realistic Simulation:** In a production environment, the model will encounter new data points with unknown distributions. By using only the training statistics, we correctly simulate how the model handles truly unseen data where only the characteristics of the training set are known.
- **Evaluation Integrity:** Fitting on the combined data leads to an "overly optimistic" evaluation. The model might appear to perform exceptionally well during development because it inadvertently used information from the test set, but it would likely generalize poorly to actual new data.

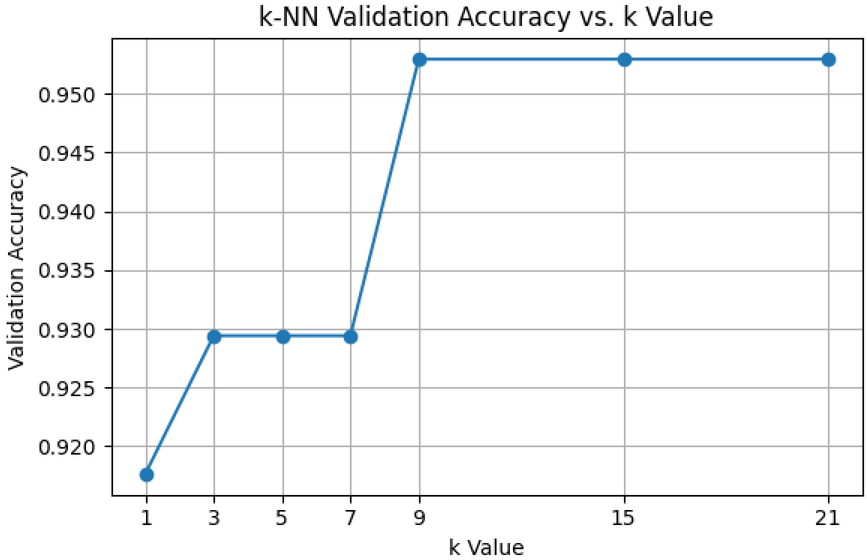
1.2 k-NN Classifier

1.2.1 Model Initialization and Hyperparameter Tuning

In the k-NN algorithm, the most critical hyperparameter is k (the number of neighbors). Selecting an appropriate *k* is essential for balancing the model: a value that is too small may lead to sensitivity to noise (overfitting), while a value that is too large may cause the model to over-generalize and overlook important patterns (underfitting).

Tuning Process: The classifier was initialized, and experiments were conducted across the specified range of values: k_values = [1, 3, 5, 7, 9, 15, 21]. For each *k*, the model was trained on the X_train_scaled data and evaluated using the X_val_scaled set to determine the optimal configuration based on accuracy.

k Value	Validation Accuracy
1	0.9176
3, 5, 7	0.9294
9, 15, 21	**0.9529**



Analysis of the Accuracy Plot:

As illustrated in the "k-NN Validation Accuracy vs. k Value" plot, accuracy improves significantly as *k* increases from 1 to 9. Starting from *k*=9, the validation accuracy reaches a plateau at **95.29%**, remaining stable through *k*=21.

Optimal k Selection: Following the assignment instructions to select the *k* value that yields the highest accuracy on the validation set, there is a three-way tie between 9, 15, and 21. For the subsequent steps of this report, ***k* = 9** is selected as the optimal value, as it is the simplest model (lowest *k*) that achieves the peak performance.

1.2.2 Final Model and Evaluation

Model Retraining and Testing

After determining that ***k* = 9** was the optimal hyperparameter using the validation set, the training and validation sets were merged to create a larger training pool of **483**

samples. This ensures the model learns from as much labeled data as possible before the final evaluation.

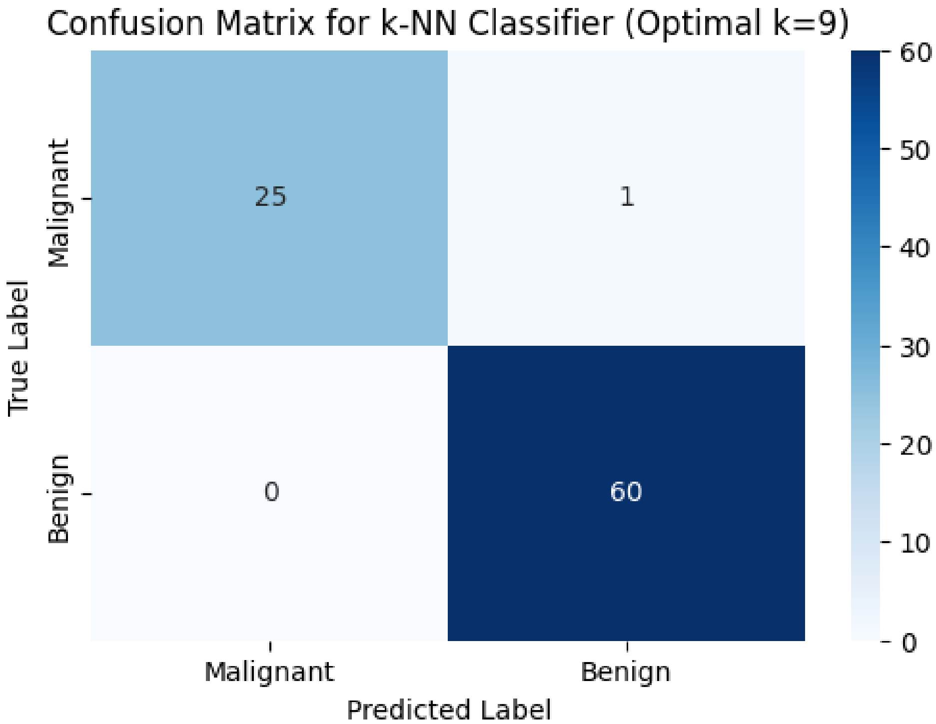
The final k-NN model was retrained on this combined dataset and then evaluated on the previously unseen **test set** (86 samples).

- **Optimal k:** 9
- **Final Test Accuracy: 98.84%**

```
Training and validation sets combined successfully.
Shape of X_combined_train_val: (483, 30)
Shape of y_combined_train_val: (483,)
```

Confusion Matrix Analysis

To move beyond simple accuracy and understand the specific types of errors the model makes, a **Confusion Matrix** was generated for the test set results.



The confusion matrix provides a detailed breakdown of the model's performance on the 86 samples of the test set. Out of these 86 cases, the model achieved an accuracy of **98.84%**, correctly predicting 85 out of 86 samples.

Breakdown of Results:

- True Negatives (Actual Malignant, Predicted Malignant): 25 cases.

- True Positives (Actual Benign, Predicted Benign): 60 cases.
- False Negatives (Actual Benign, Predicted Malignant): 0 cases.
- False Positives (Actual Malignant, Predicted Benign): 1 case.

Intuitive Explanation of Misclassification:

The model made only one mistake in the entire test set. Specifically, it misclassified one malignant tumor as benign.

Why did the model get this wrong? Intuitively, k-NN makes decisions based on "neighborhood similarity". In this specific case, the 30 numerical features (like radius, texture, and perimeter) of this malignant tumor were likely very similar to those of healthy (benign) tumors.

Imagine the 30-dimensional space where these features are plotted: this particular malignant sample was likely physically located deep inside a cluster of benign samples. Consequently, when the model looked at the 9 nearest neighbors of this patient, the majority of those neighbors were benign. Since k-NN relies on a majority vote, it "followed the crowd" and incorrectly labeled the patient as benign.

Medical Context Significance:

In a medical context, this specific error is the most critical type of mistake. Classifying a malignant (cancerous) case as benign is a False Negative for cancer, meaning a patient who actually needs treatment might be sent home without it.

1.3 Decision Tree Classifier

1.3.1 Model Initialization and Hyperparameter Tuning

Decision Trees are versatile classifiers but are highly susceptible to overfitting if the tree grows too deep or complex. To find the most effective and generalizable model, a grid search was performed over a range of structural hyperparameters.

Hyperparameter Search Space:

- **max_depth**: {2, 4, 6, 8, None} (Controls the maximum depth of the tree).
- **min_samples_split**: {2, 5, 10} (The minimum number of samples required to split an internal node).

Validation Results Summary:

The combination of all parameters resulted in 15 different configurations. The validation accuracies for these configurations are detailed in the table and visualization below:

max_depth	min_samples_split	Validation Accuracy
2	2, 5, 10	0.9176
4	**2**	**0.9647**
4	5, 10	0.9412
6	**2**	**0.9647**
6	5, 10	0.9529
8	2, 5	0.9412
8	10	0.9294
None	2, 5	0.9412
None	10	0.9294

Optimal Hyperparameter Selection: The peak validation accuracy of **96.47%** was achieved at two points: (max_depth=4, min_samples_split=2) and (max_depth=6, min_samples_split=2).

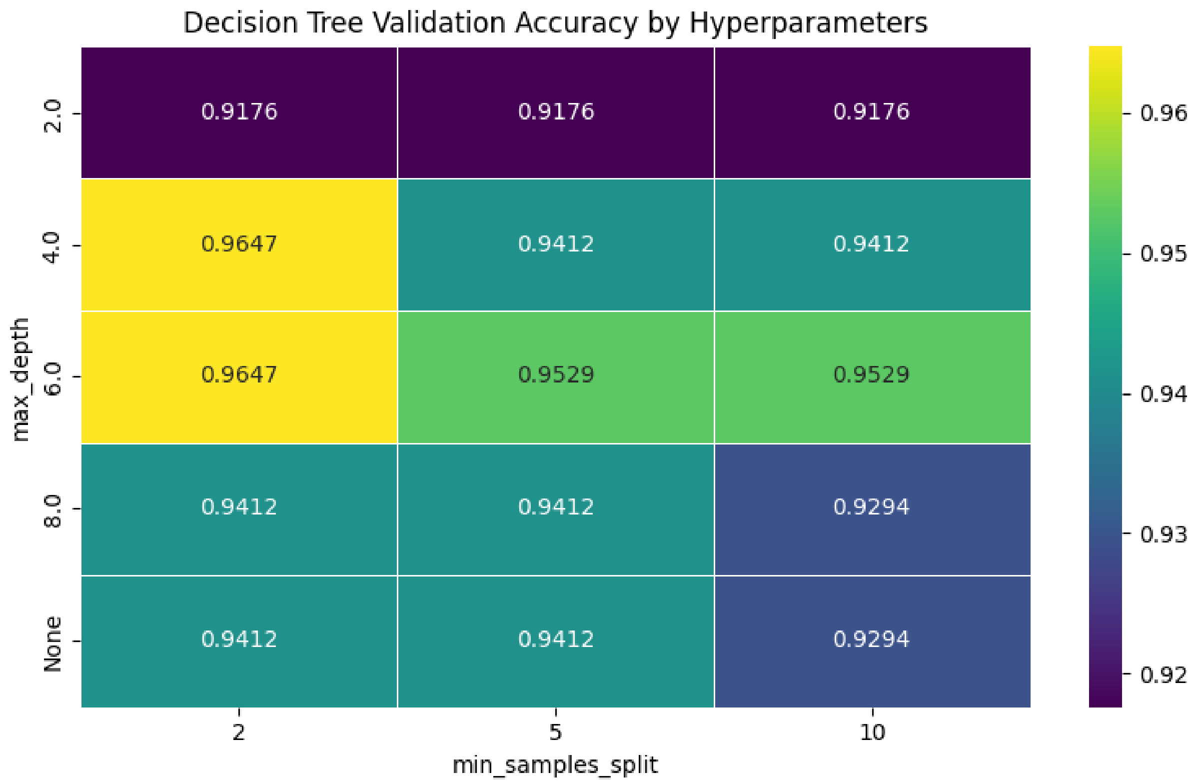
For the final model, **max_depth=4** and **min_samples_split=2** were selected as the optimal parameters.

Reasoning for Selection:

- **Accuracy:** It provides the highest recorded performance on the validation set.
- **Occam's Razor (Simplicity):** In machine learning, if two models perform equally well, the simpler one is preferred. A tree with a depth of 4 is less complex and more interpretable than a depth of 6, reducing the risk of overfitting.
- **None vs. Constraints:** As seen in the table, allowing the tree to grow without limit (max_depth=None) actually led to a decrease in validation accuracy (0.9412), confirming that unconstrained trees generalize poorly to unseen data.

Visualization Analysis:

The validation results were visualized using a heatmap (see figure below) to identify performance trends.



Interpretation of the Plot:

- X-Axis (min_samples_split): Represents the minimum samples required to split a node.
- Y-Axis (max_depth): Represents the maximum depth the tree is allowed to reach.
- Color Scale: The intensity of the color (indicated by the sidebar) represents the Validation Accuracy.
- Findings: The plot clearly visualizes that the highest accuracy (0.9647) is clustered in the top-left region where max_depth is 4 or 6 and min_samples_split is 2. As we move towards the bottom-right (deeper trees and higher split requirements), the accuracy generally decreases, indicating a loss in generalization.

- **Optimal Performance:** The highest validation accuracy of **96.47%** was achieved with two specific combinations: (max_depth=4, min_samples_split=2) and (max_depth=6, min_samples_split=2).
- **Overfitting Observation:** As the max_depth increased beyond 6 or was left unconstrained (None), the validation accuracy began to decline (dropping to

as low as 0.9294 with higher split requirements). This indicates that deeper trees were likely memorizing noise in the training set rather than learning generalizable patterns.

- **Split Impact:** Smaller min_samples_split values (like 2) generally performed better in this dataset, allowing the tree to make more granular decisions at critical junctures.

Selection of Best Hyperparameters:

Following the principle of parsimony (**Occam's Razor**), the configuration with **max_depth=4** and **min_samples_split=2** was selected as the final model. While max_depth=6 gave identical accuracy, the shallower tree (depth 4) is preferred because it is less complex, more interpretable, and offers better protection against overfitting in clinical diagnostic scenarios.

1.3.2 Final Model and Evaluation

1. Model Retraining

Following the hyperparameter tuning phase, the Decision Tree classifier was retrained using the identified optimal parameters: **max_depth=4** and **min_samples_split=2**. To leverage the maximum amount of available data, the model was trained on the combined **Training and Validation sets**, totaling **483 samples**.

2. Final Performance on Test Set

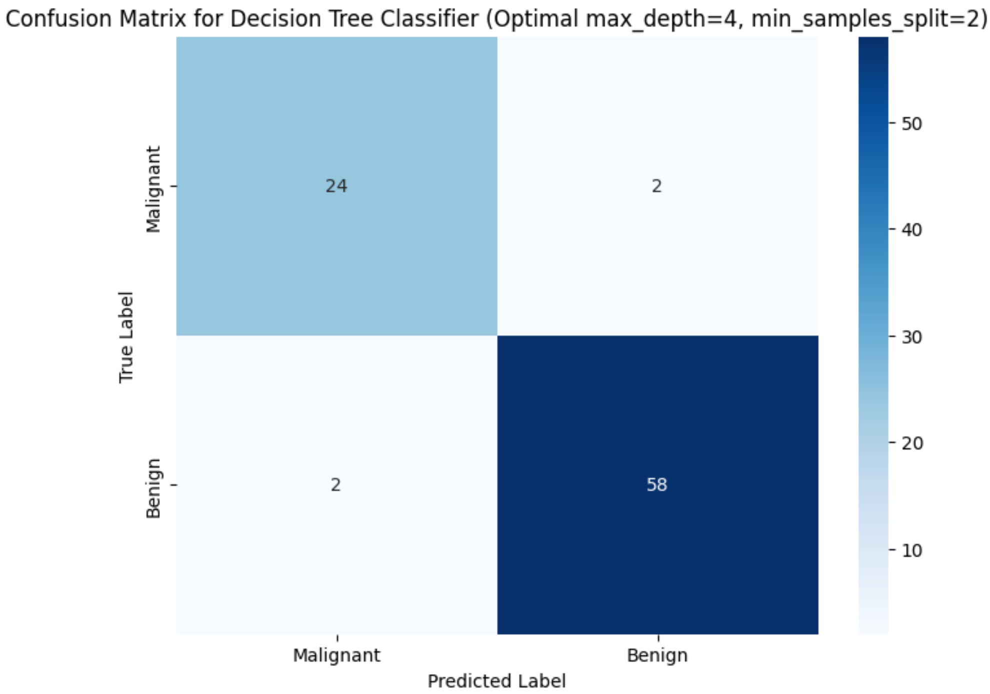
The finalized Decision Tree model was then evaluated on the independent **Test Set** (86 samples) to determine its generalization capability on unseen data.

- **Optimal Hyperparameters**: max_depth=4, min_samples_split=2
- **Final Test Accuracy**: **95.35%**

Performance Observation: The Decision Tree achieved a solid accuracy of **95.35%**. However, compared to the k-NN model's test accuracy of **98.84%**, the Decision Tree shows a slightly lower performance on this specific dataset. This suggests that the linear or distance-based decision boundaries of k-NN might be capturing the underlying patterns of the Breast Cancer dataset more effectively than the axis-parallel splits used by the Decision Tree.

3. Confusion Matrix Analysis

To evaluate the specific types of errors made by the Decision Tree, a confusion matrix was generated based on the test set predictions.



Detailed Breakdown of the Test Set (86 samples):

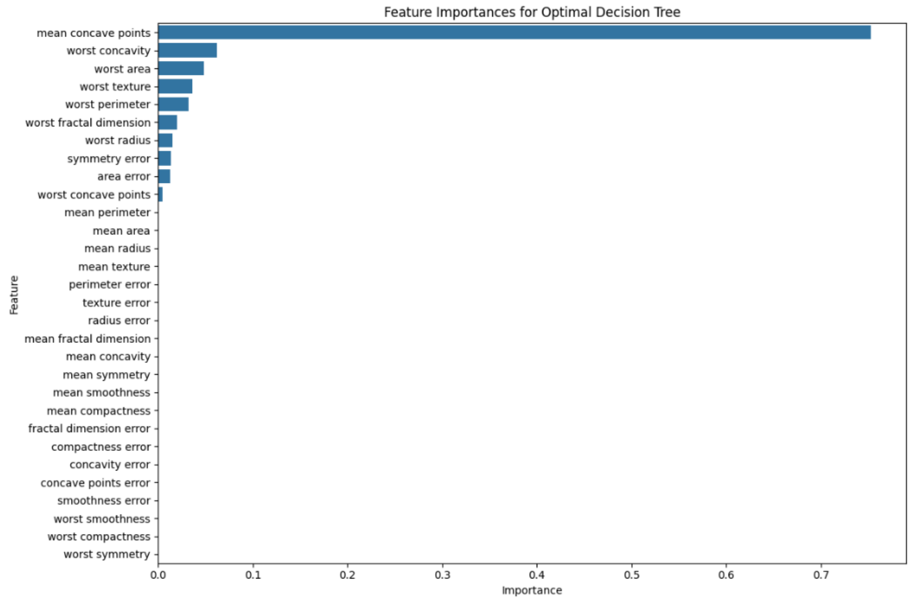
- **True Negatives (Correct Malignant):** 24
- **True Positives (Correct Benign):** 58
- **False Positives (Type I Error):** 2 cases where a benign tumor was incorrectly labeled as malignant.
- **False Negatives (Type II Error):** 2 cases where a malignant tumor was missed and labeled as benign.

Medical Context Discussion:

The model produced **2 False Negatives**. In breast cancer diagnosis, this is a critical concern as it represents missed cancer cases, potentially delaying life-saving treatment. While the **2 False Positives** cause unnecessary patient anxiety and further invasive procedures (like biopsies), they are generally considered less severe than missing a malignancy entirely.

4. Feature Importance Analysis

One of the key advantages of Decision Trees is interpretability. The feature_importances_ attribute was used to identify which physical characteristics were most influential in the model's decision-making process.



Most Informative Features:

1. **Mean Concave Points:** This is by far the most dominant feature, contributing over **70%** to the tree's splits.
2. **Worst Concavity:** The second most influential feature.
3. **Worst Area & Worst Texture:** These follow as secondary but significant indicators.

Analysis: The heavy reliance on "Mean Concave Points" suggests that the presence and number of indentations in the cell nuclei are the strongest indicators of malignancy for this model. Most other features have near-zero importance, indicating that the Decision Tree was able to achieve its **95.35%** accuracy by focusing on a very small subset of highly discriminative features.

2. Report - Final Sections

2.1 Misclassification Analysis

This section compares the error profiles of the k-Nearest Neighbors (k-NN) and Decision Tree models to determine their clinical suitability.

Metric	k-NN (k=9)	Decision Tree (d=4,s=2)
False Positives (Type I)	1	2
False Negatives (Type II)	**0**	**2**
Total Errors	1	4

Discussion of Criticality: In the medical context of breast cancer diagnosis, **False Negatives (FN)** are significantly more critical than False Positives (FP). A False Negative occurs when a malignant tumor is missed and incorrectly classified as benign, which can lead to delayed treatment and life-threatening progression of the disease.

- **k-NN Performance:** The k-NN model achieved an exceptional result with **zero False Negatives**, meaning it did not miss a single malignant case in the test set.
- **Decision Tree Performance:** The Decision Tree produced **two False Negatives**. While this is a small number, any missed malignancy is a serious concern in clinical practice.

2.2 Conclusion

The evaluation of k-NN and Decision Tree classifiers on the Breast Cancer Wisconsin dataset reveals a clear distinction in performance and utility.

1. Performance Comparison:

- **Accuracy:** The **k-NN model** outperformed the Decision Tree, achieving a test accuracy of **98.84%** compared to **95.35%**.
- **Reliability:** k-NN demonstrated superior robustness in minimizing critical errors, identifying all malignant instances correctly, whereas the Decision Tree was slightly less reliable in this regard.

2. Interpretability vs. Performance:

- **Decision Tree:** Offers higher interpretability. The feature importance analysis clearly identified **'mean concave points'** as the primary driver of its predictions, providing a "reason" for the diagnosis.
- **k-NN:** Operates as a "black box" regarding feature importance, as it relies purely on spatial proximity in the feature space.

Final Verdict: While the Decision Tree provides valuable insights into feature significance, the **k-NN model (*k*=9)** is the preferred model for this specific task. Its ability to achieve higher overall accuracy and, most importantly, its **perfect record in avoiding False Negatives** makes it a safer and more effective diagnostic tool for breast cancer detection